high Level
c = a + b

Assembly
add ax, bx

Low-level
01011111101

Source code → Assembly → machine code

Source code
↓
Program that is written by humans e.g in CPP / Python

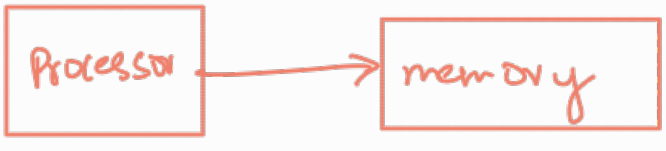
machine code
↓
what computer understands.
(0 / 1's)

why use assembly Language
→ efficient + we can see code from more closer.
→ Assembly lets us free memory space

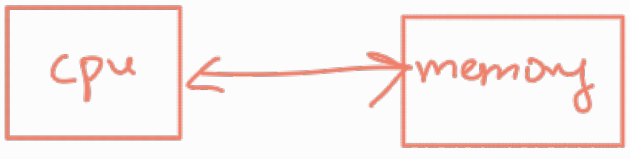
Buses (not part of CPU)

1) Address Bus:. The cpu uses address Bus to select a specific location (Address) for finding / storing data

Uni-direction



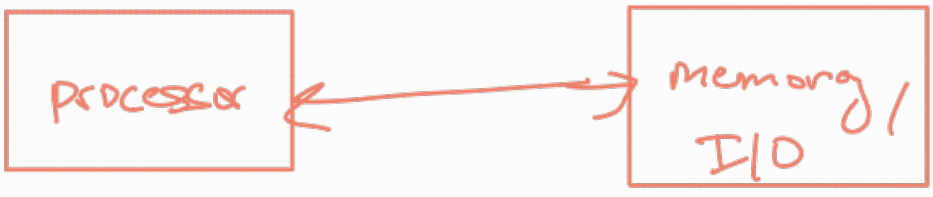
2) Data Bus : Moves data between the memory → in read operation and cpu.
cpu → memory in write operation.



Bi-directional

3) Control Bus: Through the help of this Bus the cpu gives commands to the rest of the computer.

e.g used to inform memory about whether to read/write operation.



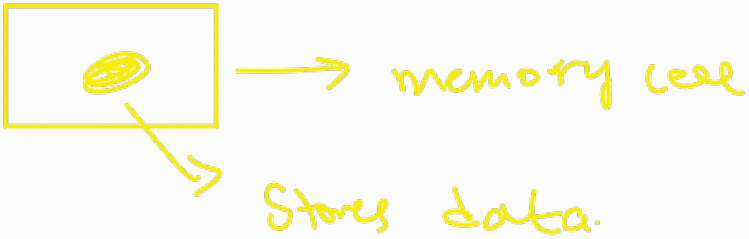
Bi-directional

Memory is itself dumb and doesn't know what to do with data.

Control Bus is complex

Data Bus is a private Bus owned by the CPU and to bring data from outside the CPU uses control Bus.

Cell-width:: How much data (value) a memory cell can store one at a time (no of bits it can store)



word size of memory :: how many bits the memory can send to the CPU at a time

cell width = data Bus width = faster system.

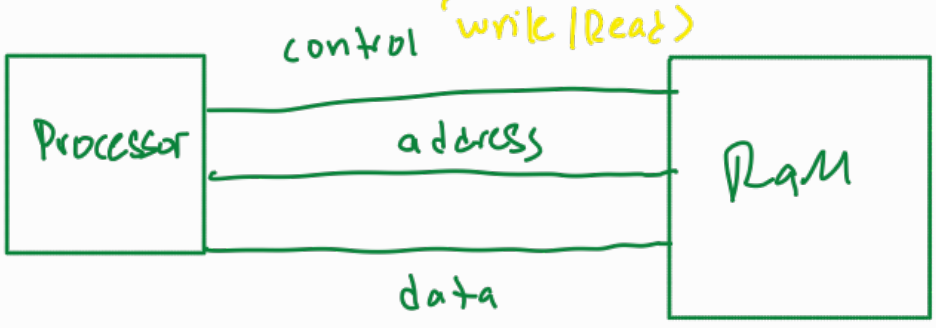
control Bus

↳ makes synchronization b/w the cpu and the memory inorder to ensure communication.

why bi-directional

Because Memory is dumb (cannot do anything on its own)

If memory is locked and control Bus is used to inform the cpu, Moreover control Bus is used in connecting I/O with cpu.



Registers: (part of cpu)
Temporary storage spaces.

operand: Data on which we perform certain operation.

5+3
↳ operand.

opcode: no. that tells the cpu which operation to perform.

ADD (human understandable) ⟷ 123 machine equivalent

Instruction groups (category of commands stored in cpu)

1) Data Movement ⇒ mov ax, 5

2) Arithematic Logic ⇒ add ax, bx | and ax, bx
add bx, [1200]

3) Program control ⇒ cmp ax, 0 | jne 0x1234
(comparisions)

4) Special Instruction ⇒ sti | cli (flags)

↓

changes the behavior of cpu.

cli instruction closes the cpu from outside world and do a more imp work at hand

sti → opens the cpu to outside world

Intel 8088 architecture

↳ design of how the cpu is built and how it works inside and the intructions that are valid for it

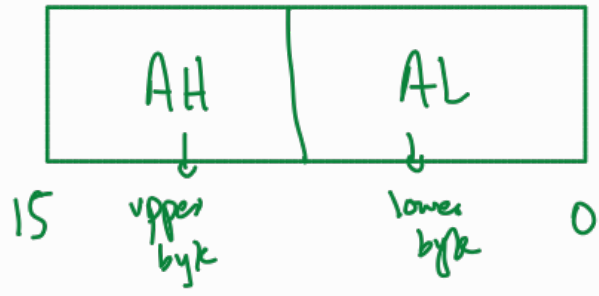
Architecture Size = Size of Ax (Accumulator)

16 bit Architecture means our registers are of 16 bits

general purpose Registers:

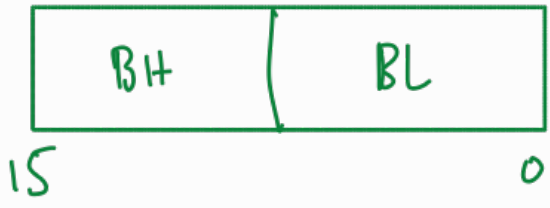
Ax, Bx, Cx, DX

dividing the register into half as they are limited and don't want to use them at once

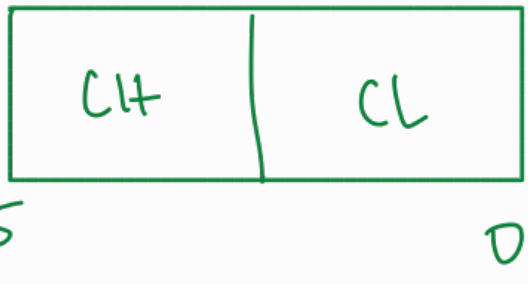


⇒ Ax

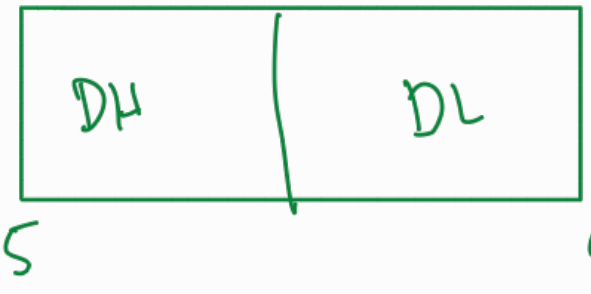
(Any change in AH/AL will affect Ax)



⇒ Bx



⇒ Cx



⇒ Dx

Ax → Accumulator
(used in logical / arithmatic)

Bx → Base
(used in memory / offset)

Cx → counter
(Looping)

Dx → Destination
(works for I/O)

general purpose : CPU can use them for diff things

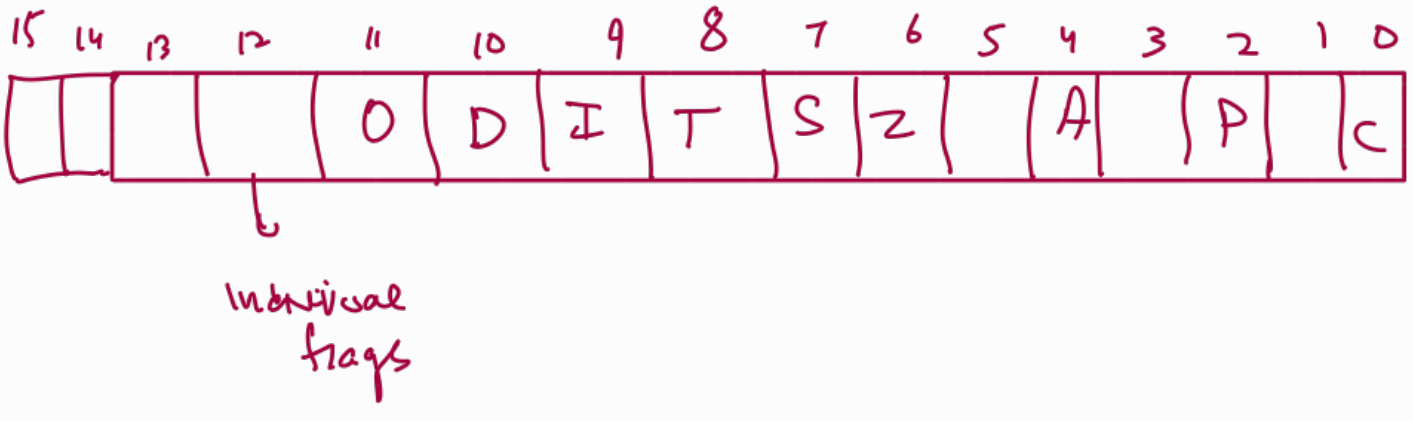




AX = (1283) in decimal

Flag Register → Like a report card to CPU. After if some result happened it tells the CPU. e.g 5-5:0 if result =0 Set flag → 1

they are used bitwise



Carry flag → Stores the overflow bit. e.g adding two 8 bit numbers together → 9 bit Answer.

↳ Carry | Borrow happens leftmost Bit.

carry flag eg
$$\begin{array}{r} 1100 \\ +\ 0100 \\ \hline 0000 \end{array}$$

$$\begin{array}{r} 0011 \\ -\ 0100 \\ \hline 1111 \end{array}$$

Parity flag → Parity is no of 1s in a binary number. → even | odd

even Parity so flag = 1 and vice versa.

(used in maintaining data integrity b/w Sender | reciever)

Auxilary Carry

$$\begin{array}{r} \overset{1}{\text{XXXX}}\ \text{XXXX} \\ +\ \text{YYYY YYYY} \\ \hline \end{array}$$

Turns on if there is a carry | Borrow in between the nibbles.

Not the Leftmost, Rightmost bit.

$$\begin{array}{r} \overset{0}{0}00X\ \overset{1}{0}\overset{1}{0}\overset{1}{0}0 \\ -\ 0000\ 0001 \\ \hline 0000\ 1111 \end{array}$$ (Borrowed)

$$\begin{array}{r} \overset{1}{F}\ \overset{1}{F}\ F\ 2 \\ +\ 0\ 0\ 1\ 3 \\ \hline (1)\ 0\ 0\ 0\ 5 \end{array}$$

Hexadecimal Addition

F + A = 15 + 10 = 25

Subtract 16

25 − 16 = 9

$$\begin{array}{r} F \\ +\ A \\ \hline 9 \end{array}$$ with carry 1

Subtracting 16 represent 1 carry.

Zero flag is Set to 1 if the result of last math | Logic operation is 0

Sign flag (works only for numbers that are represented in 2's complement).

Sets flag to 1 if MSB = 1 (neg)

Sets to 0 if MSB = 0 (pos)

Trap flag used in debugging

Interrupt flag :: A programmer sets this flag

1 → Interrupts are allowed. (cpu Stops doing whatever it was currently doing and handles Interrupt

0 → dont allow Interrupts (meaning give all resources of cpu to programmer) Disconnect from outside world

Direction flag → controls direction of memory access / moving through memory (particularly when processing string)



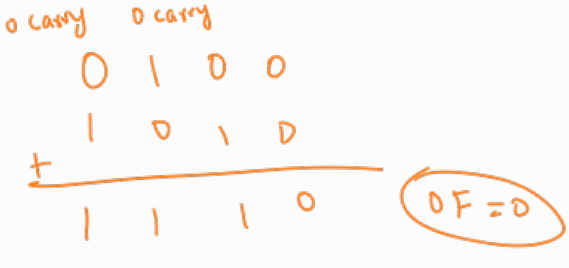
D = 0 (Direc. of accessing memory)

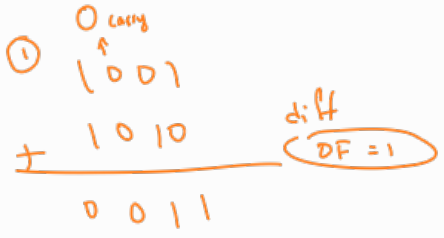
D = 1

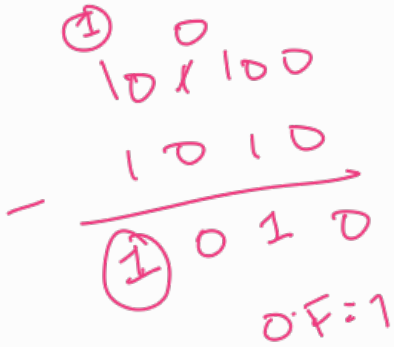
overflow flag → if at MSB carry in carry out diff then overflow = 1

Same with borrow in borrow out

Looping (out of Range)







Index Registers

SI

DI

Segment Registers

CS, DS, SS, ES

Pointer Registers

SP

BP

IP

Our first program.

move 5 to ax

5 ↓ Source

ax ↪ destination

Adding three numbers.

```
mov   ax, 5      (load 5 into ax)
mov   bx, 10     (load 10 into bx)
add   ax, bx     (accumulate the sum in ax)
mov   bx, 15     (load third no (15) into bx)
add   ax, bx     (accumulate sum in ax)
```

terminated the program

; this is a comment in assembley language

Segmented memory model

→ cpu is very fast
→ Ram is much slower
→ if cpu had to go to Ram for every single operation, it would waste alot of time,

hence cpu has Registers → Data/Instructions are fetched from RAM into Registers → cpu.

Problem

8086 cpu is a 16bit, so Reg are 16bits
16 bit registers can hold numbers up to 64KB.
Intel wanted to access memory of 1MB (20 bits)
How can 16bit registers handle 20 bit addresses.

Trick → Segmentation

Instead of one register they used two Registers.

Logical address = Segment : offset

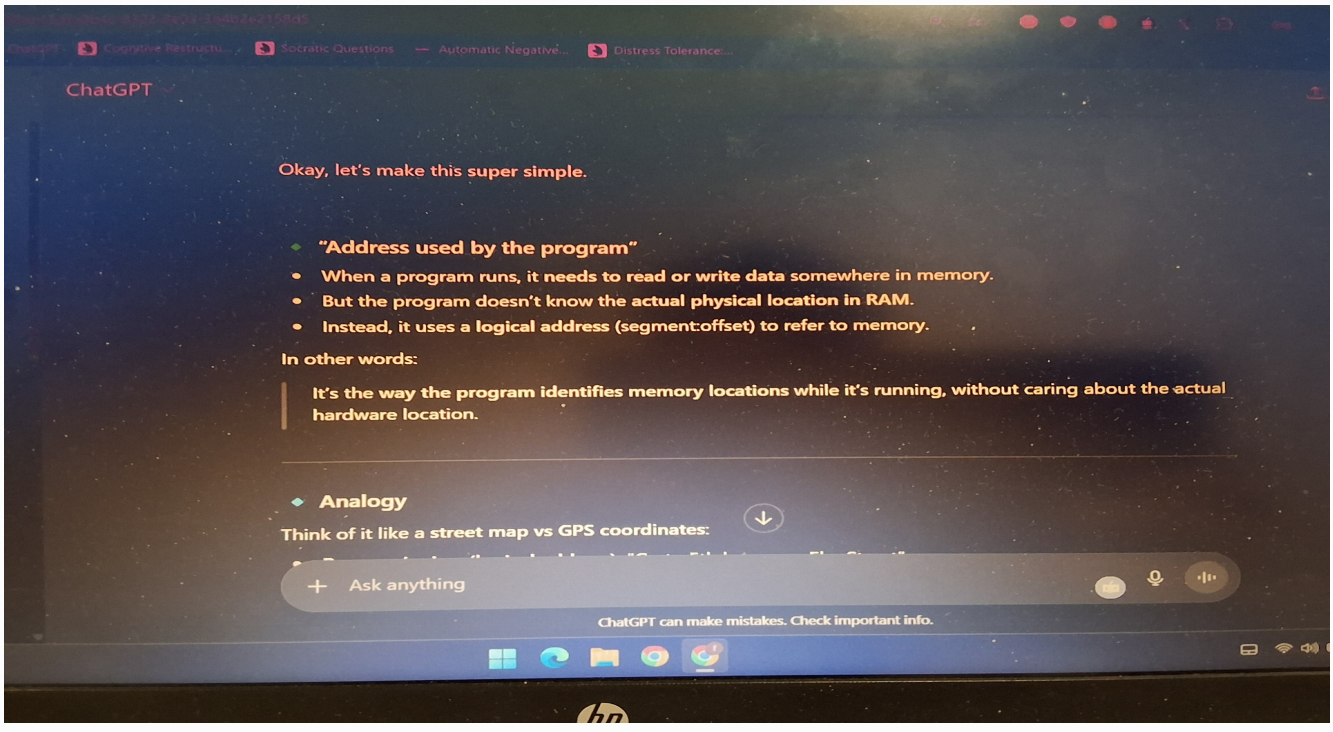
address used by program

Physical address = (Segment × 16) + offset
(calculated from logical address)

Special purpose

Segment Register : tells cpu where the window starts
offset Register : tells cpu how far inside that window to go.
(means moving to the next instruction within the same window)

general purpose



these are Seg Reg

Segment :: CS, DS, ES, SS (16 bit)

offset :: IP, BP, SP, BX, SI, DI (16 bit)

> generates 20 bit address.

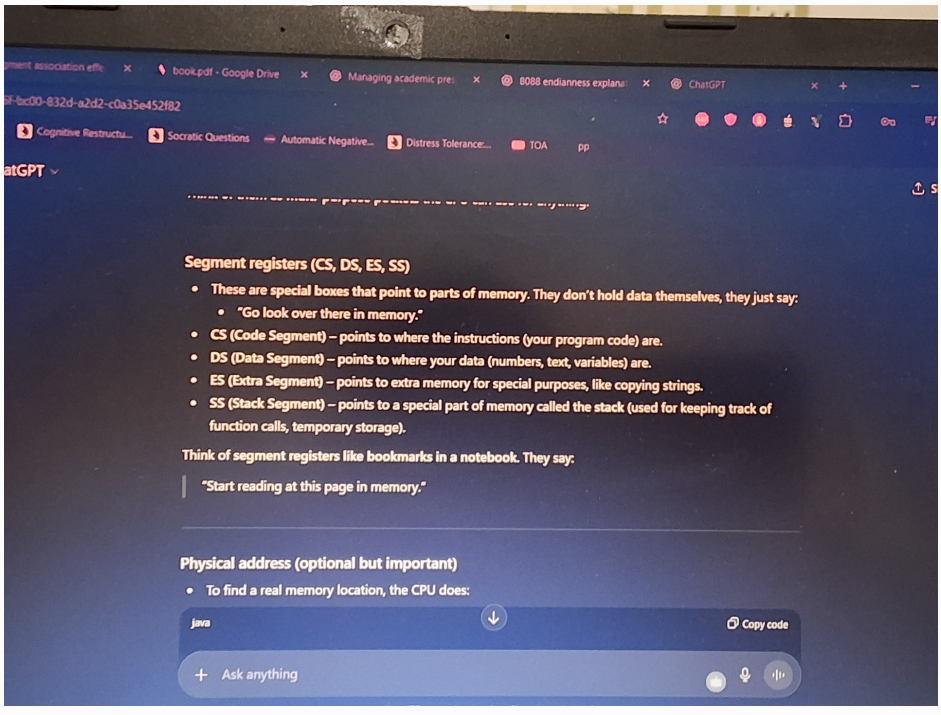
The cpu can look through four windows of the Segment Registers

CS → window for instruction
DS → window for data
ES → window for extra data
SS → window for stack op

Each of these four Segment Register hold one base address for cpu to access

and The cpu can open all four windows at the same time and switch b/w the windows depending on the instruction that is being executing

Each Segment window is of fixed size of 64KB


book.pdf - Google Drive
Managing academic pres
8088 endianness explana
ChatGPT
Cognitive Restructu...
Socratic Questions
Automatic Negative...
Distress Tolerance:...
TOA
pp
atGPT
Segment registers (CS, DS, ES, SS)
These are special boxes that point to parts of memory. They don't hold data themselves, they just say:
"Go look over there in memory."
CS (Code Segment) – points to where the instructions (your program code) are.
DS (Data Segment) – points to where your data (numbers, text, variables) are.
ES (Extra Segment) – points to extra memory for special purposes, like copying strings.
SS (Stack Segment) – points to a special part of memory called the stack (used for keeping track of function calls, temporary storage).
Think of segment registers like bookmarks in a notebook. They say:
"Start reading at this page in memory."
Physical address (optional but important)
To find a real memory location, the CPU does:
java
Copy code
Ask anything


labels → linear memory ka hisaab

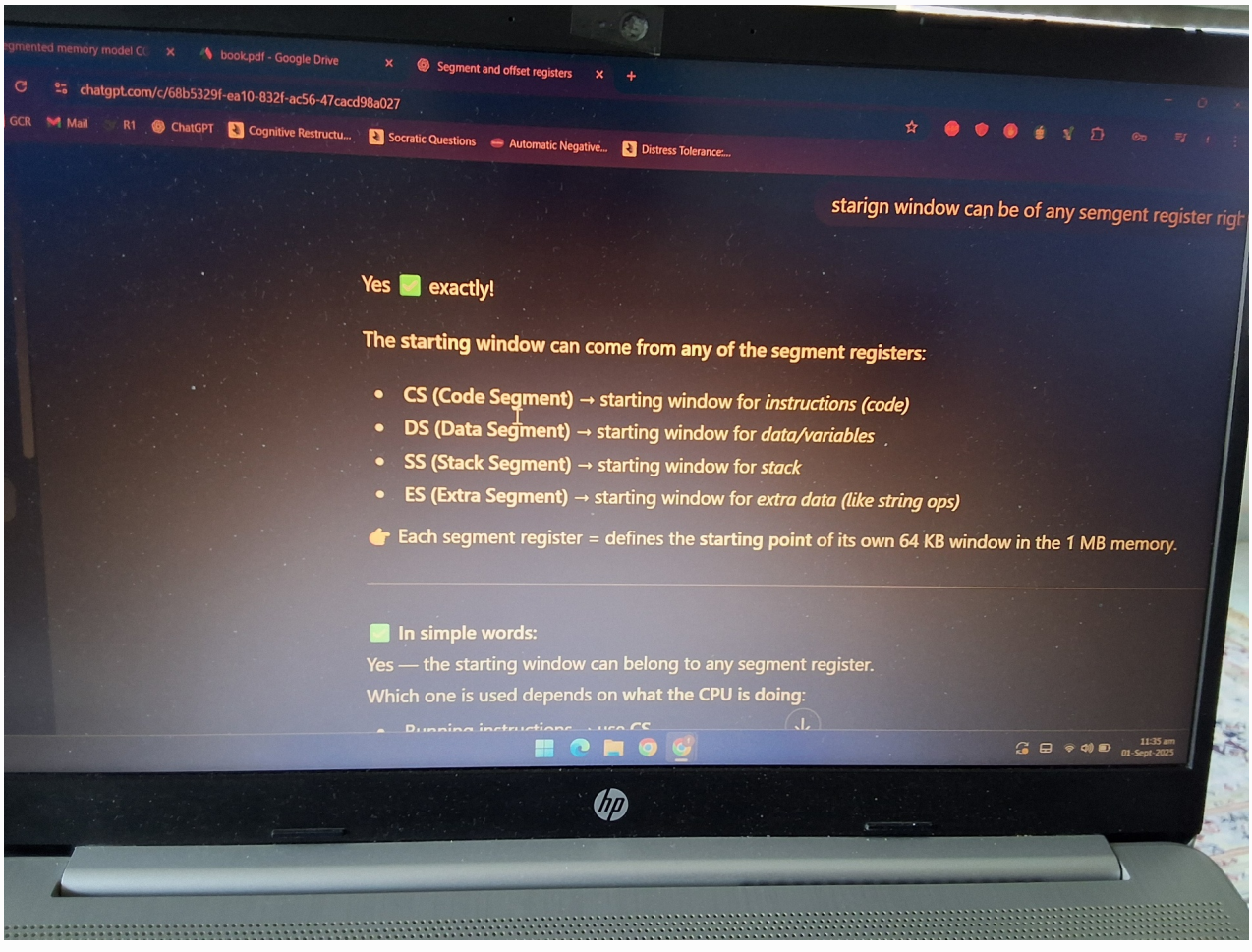
offset :- IP, BP, SP, BX, SI, DI

IP (Instruction pointer) It can count upto 64K only and cannot reach the whole 1MB.

IP only attaches to CS window. Because IP tells the cpu which instructn to execute. Instructions always live in CS.

And the CS window can move within itself for IP to look for instructions.

Note:- Starting window can be of any segment Register.



Calculating physical address

IP: 0100
CS: 1DDD

A - 10
B - 11
C - 12
D - 13
E - 14

Physical address = (1DDD × 16) + 0100

= 1DDD0 + 0100

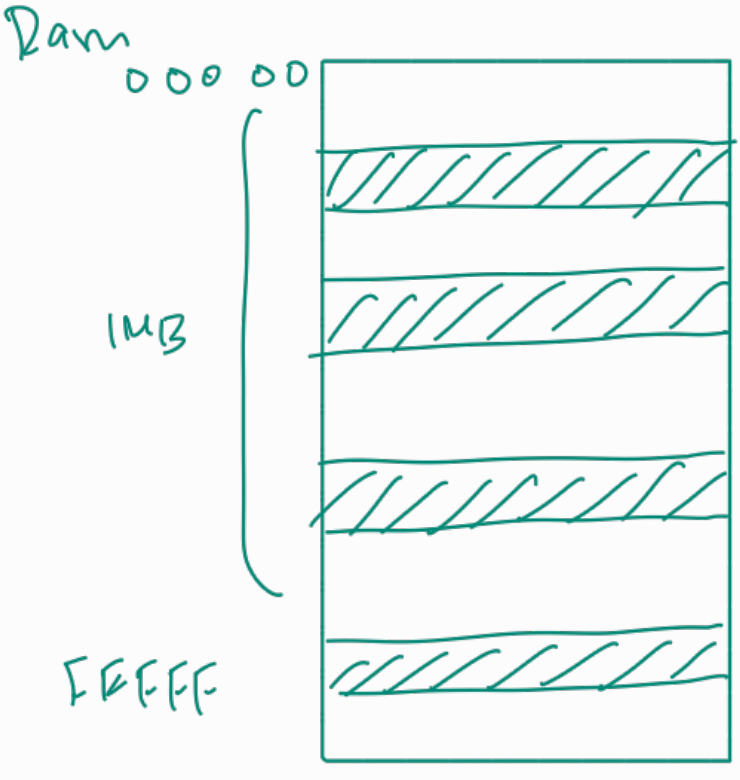
$$\begin{array}{r} 1DDD0 \\ +\ 0100 \\ \hline 1DED0 \end{array}$$

=

(1DED0) final Answer

Multiplying 16 with a hex no is just adding 0 to the Right most.

$123 \times 256 \rightarrow 123 \times 16^2 \Rightarrow 12300$



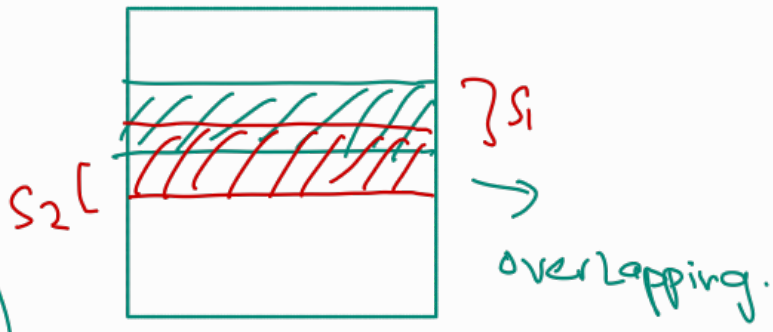


Seg Reg → Seg ↑ offset Looks deep down

overlapping Segment

1DDD : 0100 → 1DED0
1DED : 0000 → 1DED0

↳ Diff Segment windows covering each other

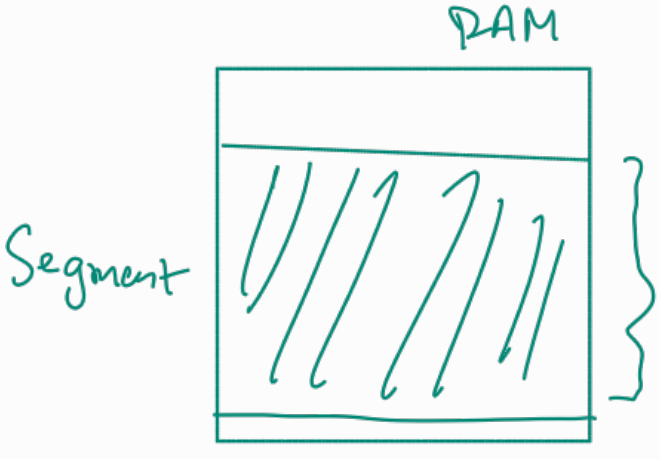


Both point to the same physical location.

Now if you modify anyone of the segment windows it will automatically change the other address.

and Both will remain overlapping if we change one of them.

end of seg = Start address + FFFF



0000 Offset Starting. (first address in window)
0001 (Second address)

FFFF Offset

At max 4 Segments can overlap with each other

Paragraph Boundary:

First Segment address:: 0000
offset:: 0000

(0000x16) + 0000
→ 00000

Address of a Segment is always multiple of 16.

Second Segment address :. (0001 x 16) + 0000

→ 00010

Difference of First Seg and last Seg is of 16bytes.

So a new Segment only start at these 16-byte steps never in between. these are know as paragraph Boundaries.

Address wraparound:

Max address in memory is of 1MB.

When we calculate the physical address using the formula

Sometimes it results in bigger than 1MB

The extra bits are thrown away and the address comes back to the starting location / beginning of the memory.

e.g

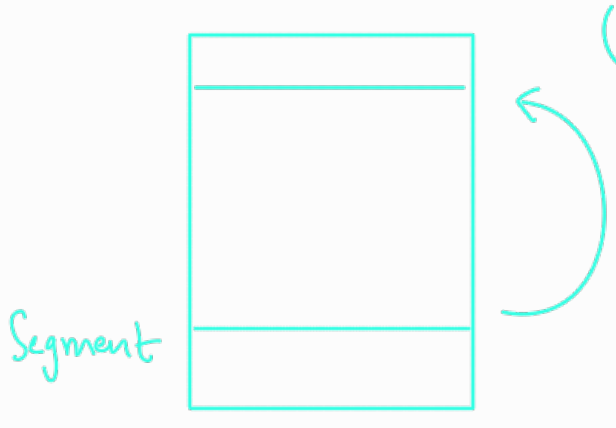
CS: FFFFh IP: FFFFh

(FFFFh x 16) + FFFFh = FFFF0h + FFFFh

```
 1 1 1
 F F F F 0h
   F F F F h
-----------
 1 0 F F E F h
```

→ 10FFEFh → (CPU will drop the extra carry bit.)

→ 0FFEFh



A → 10
B → 11
C → 12
D → 13
E → 14
F → 15

Q:. is 55555 a valid Segment address → (value in seg reg.)

no because max a Segment can store FFFFh.

Our first program

FFFF → 16bit / 64kB

FFFF → 20bit / 1MB (max address)

move 5 to ax

5 ↓ Source

ax ↪ destinal

Adding three numbers:

org [0x0100] → offset / start from this address

mov ax, 5 (load 5 into ax)

mov bx, 10 (load 10 into bx)

add ax, bx (accumulate the sum in ax)

mov bx, 15 (load third no (15) into bx)

add ax, bx (accumulate sum in ax)

(Segment is only needed in memory access)

(Default Seg is CS)

terminated the program

mov ax 0x400

int 21h

where result will go

operand

Instruction

operation

operation

operation

operation

Destination, Source → where data come from

Destination

Source

mov ADDelc

hexadecimal representations

→ 0x143 143

→ 4Fh 4F

converting Assembly to machine code:

5 in 16bit is
0000000000000101

MOV Ax, 5 → opcode

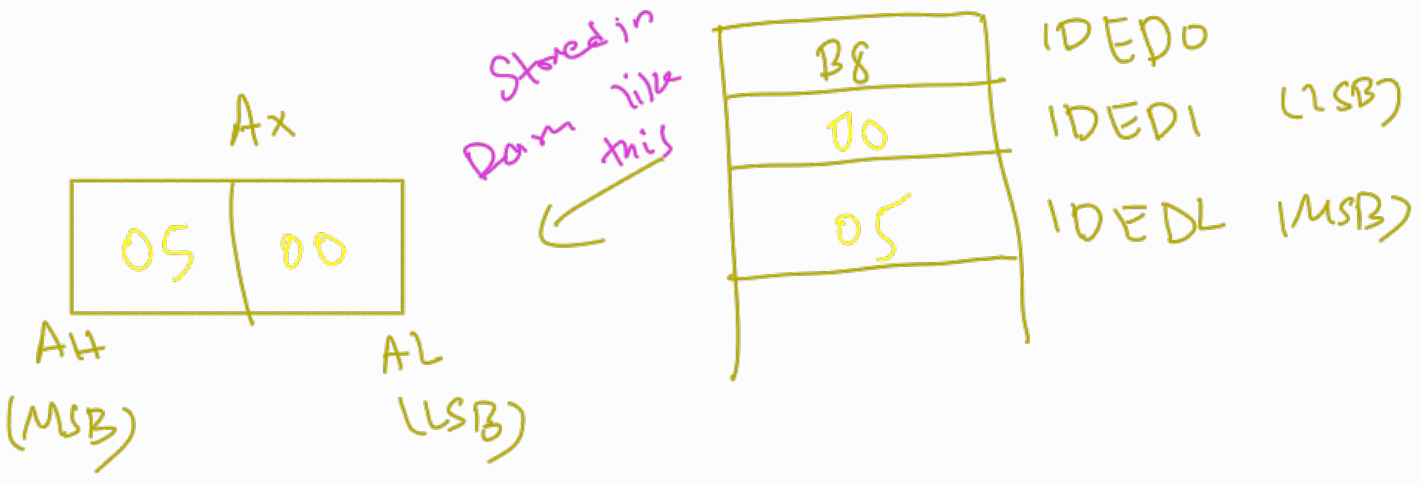
B80500 → opcode

ADD Ax, Bx → opcode

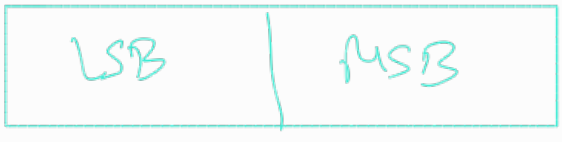
01 D8 → opcode

1byte = 8bits

little endian notation



Little endian notation

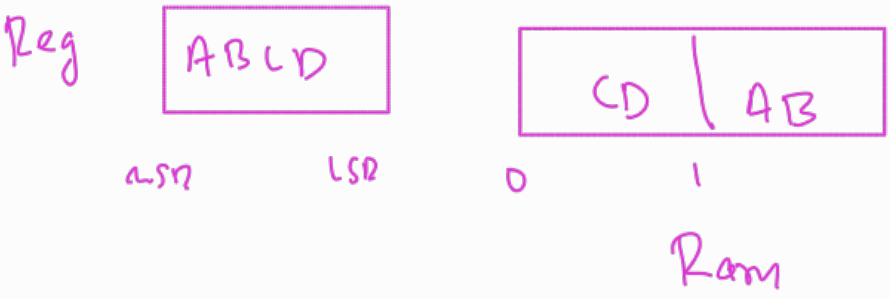


Lesser address Higher address

computer Storing (in RAM) ?

Store the LSB in Lower address and MSB in higher address

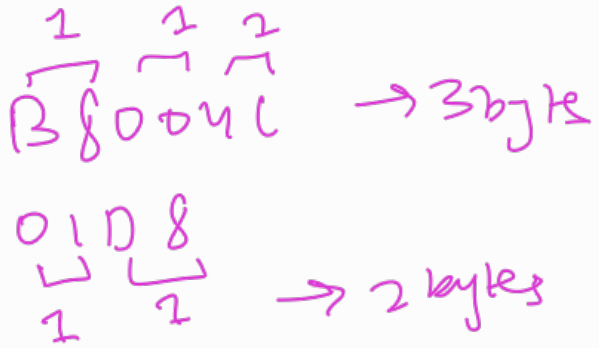
Big endian Store MSB in Lesser address and LSB in Higher address



(little indian) notation.

1 hexa digit = 4 bits

Q What is the total size of listing file = total no of bytes in machine code



= 5 byte size of listing file

Smallest unit in Ram is 1 byte.